📘 Module: ANSI SQL using MS SQL Server

1️⃣ Overview of ANSI SQL

🔹 What is ANSI SQL?

ANSI SQL is the standardized version of SQL defined by:

👉 American National Standards Institute

It ensures uniformity across database systems such as:

- Microsoft SQL Server
- MySQL
- Oracle Database
- PostgreSQL

🔹 ANSI SQL vs T-SQL (Important for Intermediate Level)

SQL Server supports ANSI SQL + its own extension called **T-SQL (Transact-SQL)**.

ANSI SQL	T-SQL (SQL Server Specific)
Standard syntax	Microsoft extension
Portable	SQL Server only
Basic querying	Advanced programming features

Example:

ANSI Standard:

```
SELECT * FROM Students;
```

T-SQL specific:

```
SELECT TOP 5 * FROM Students;
```

Intermediate learners must understand portability vs vendor-specific features.

2️⃣ Introduction to Database Design

🔹 What is Database Design?

Database design is the systematic process of organizing data into tables to:

- Eliminate redundancy
- Maintain integrity
- Improve performance
- Ensure scalability

🔹 Why Poor Design is Dangerous

Poorly designed databases lead to:

- Duplicate data
- Update anomalies
- Delete anomalies
- Slow performance
- Complex maintenance

🔹 Example: Poor vs Good Design

❌ Poor Design (Single Table)

OrderID CustomerName ProductName Price

Customer name repeats multiple times.

✅ Good Design (Normalized)

Customers Table
Products Table
Orders Table
OrderDetails Table

This reduces redundancy and improves maintainability.

3️⃣ Data Modelling Process

Data modeling is done before writing SQL code.

It answers:

- What data?
- How related?
- What constraints?

🔹 Step 1: Identify Entities

Entity = Real-world object.

Examples:

- Customer
- Order
- Product
- Employee

🔹 Step 2: Identify Attributes

Customer:

- CustomerID
- Name
- Email
- Phone

🔹 Step 3: Identify Relationships

1-to-Many

One Customer → Many Orders

Many-to-Many

Students ↔ Courses
(Requires junction table)

🔹 Types of Data Models

1️⃣ Conceptual Model

Only entities + relationships
(No keys or data types)

2️⃣ Logical Model

Includes:

- Attributes
- Primary keys
- Relationships

(No database-specific data types yet)

3️⃣ Physical Model

Includes:

- SQL Server data types
- Indexes
- Constraints
- Identity columns

Example:

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY IDENTITY(1,1),
```

```
    Name NVARCHAR(100) NOT NULL,
    Email NVARCHAR(100) UNIQUE,
    CreatedDate DATETIME2 DEFAULT SYSDATETIME()
);
```

4️⃣ Database Design (Intermediate Depth)

🔹 Step-by-Step Design Approach

1️⃣ Requirement Analysis

Understand:

- What reports are needed?
- Expected data volume?
- Future scalability?

2️⃣ Apply Normalization

1NF

No repeating groups.

2NF

No partial dependency.

3NF

No transitive dependency.

Example: 3NF Violation

| StudentID | StudentName | CourseName | CourseFee |

CourseFee depends on CourseName, not StudentID.

Solution:
Create separate Course table.

5️⃣ Design Considerations (Intermediate Level)

🔹 1. Data Integrity

Use constraints:

PRIMARY KEY

FOREIGN KEY

UNIQUE

CHECK

NOT NULL

DEFAULT

Example:

Age INT CHECK (Age >= 18)

🔹 2. Performance Considerations

- Use proper indexing
- Avoid unnecessary VARCHAR(MAX)
- Choose smallest possible data type
- Avoid over-normalization in reporting systems

Example:

CREATE INDEX IX_Customer_Email ON Customer(Email);

🔹 3. Scalability

Plan for:

- Growing data
- Archival strategies

- Partitioning large tables

🔹 4. Security

- Role-based access
- Column-level security
- Avoid storing plain-text passwords

🔹 5. Naming Conventions

Good practice:

- Singular table names
- Clear column names
- Consistent casing

Example:
CustomerID (not custid or CID)

6️⃣ Data Types in SQL Server (Beginner → Intermediate Depth)

Choosing the correct data type impacts:

- Storage size
- Performance
- Index efficiency
- Data accuracy

🔹 Numeric Types

Type	Use Case
INT	Standard IDs

Type	Use Case
BIGINT	Large-scale systems
DECIMAL(p,s)	Money values
FLOAT	Scientific calculations

Use DECIMAL for financial data (never FLOAT).

- **Character Types**

Type	Use
CHAR(n)	Fixed length
VARCHAR(n)	Variable
NVARCHAR(n)	Unicode

If multilingual support is required → use NVARCHAR.

- **Date/Time Types**

Type	Use
DATE	Date only
DATETIME	Old standard
DATETIME2	Recommended

Prefer DATETIME2 for new systems.

- **Special Types**

Type	Usage
BIT	Boolean
UNIQUEIDENTIFIER	GUID
XML	Structured XML data
VARBINARY	File storage

🎯 **Real-World Example (E-Commerce Mini Design)**

```
CREATE TABLE Product (
   ProductID INT PRIMARY KEY IDENTITY(1,1),
   Name NVARCHAR(150) NOT NULL,
   Price DECIMAL(10,2) NOT NULL,
   Stock INT CHECK (Stock >= 0)
);

CREATE TABLE Customer (
   CustomerID INT PRIMARY KEY IDENTITY(1,1),
   Name NVARCHAR(100),
   Email NVARCHAR(100) UNIQUE
);

CREATE TABLE Orders (
   OrderID INT PRIMARY KEY IDENTITY(1,1),
   CustomerID INT,
   OrderDate DATETIME2 DEFAULT SYSDATETIME(),
   FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
```

);

This example includes:

- Primary Keys
- Foreign Keys
- Identity columns
- Constraints
- Proper data types
- Default values